

INFORME CON DOCUMENTACIÓN DEL PROCESO ETL

NOMBRE Y APELLIDO DE LOS ALUMNOS

Barboza Virginia
Cabana Enzo
Delfini Donata V.
Gagliardi Verónica A.

MATERIA

Módulo “Innovación y gestión de datos”

COHORTE 2025

Proceso ETL para Análisis de Datos de Ventas y Clientes

Descripción detallada de las operaciones de extracción, transformación y limpieza realizadas en los datos de ventas y clientes.

1. INTRODUCCIÓN

1.1. Objetivo del trabajo práctico

El objetivo central de este trabajo práctico es adquirir experiencia práctica en el proceso ETL (Extracción, Transformación y Limpieza) al integrar y preparar datos de ventas y clientes para su análisis.

1.2. Recursos utilizados

1.2.1. Datasets: `customer_data.csv` y `sales_data.csv` del conjunto "Sales and Customer data" (obtenidos de Kaggle).

1.2.2. Herramientas: Python con las librerías Pandas (para ETL) y(para conexión a BD), Jupyter Notebook, y MariaDB/Similar (para la base de datos relacional).

2. PROCESO ETL CON PANDAS (Extracción,Transformación, Carga)

2.1. Extracción de datos

2.1.1. Carga de datos en dataframes, extracción y acceso: en primer lugar se importó la librería Pandas utilizando el comando *import pandas as pd*, luego se cargaron los archivos en memoria creando dos dataframes: *df_clientes* y *df_ventas*. Y para validar la extracción, en cada caso, se usa el método *df_clientes.head()* y *df_ventas.head()* que muestra las primeras 5 filas de cada uno, evitando así saturar el Notebook, ya que los dataFrames pueden tener miles de filas.

Jupyter SalesandCustomerData Last Checkpoint: 1 hour ago

File Edit View Run Kernel Settings Help

JupyterLab Python [conda env:base] * Anaconda Toolbo

```
[2]: import pandas as pd
[24]: df_clientes = pd.read_csv('customer_data.csv')
[25]: df_clientes.head()
```

	customer_id	gender	age	payment_method
0	C241288	Female	28.0	Credit Card
1	C111565	Male	21.0	Debit Card
2	C266599	Male	20.0	Cash
3	C988172	Female	66.0	Credit Card
4	C189076	Female	53.0	Cash

Jupyter SalesandCustomerData Last Checkpoint: 1 hour ago

File Edit View Run Kernel Settings Help

JupyterLab Python [conda env:base] * Anaconda Toolbox

4 C189076 Female 53.0 Cash

```
[26]: df_ventas = pd.read_csv('sales_data.csv')
[27]: df_ventas.head()
```

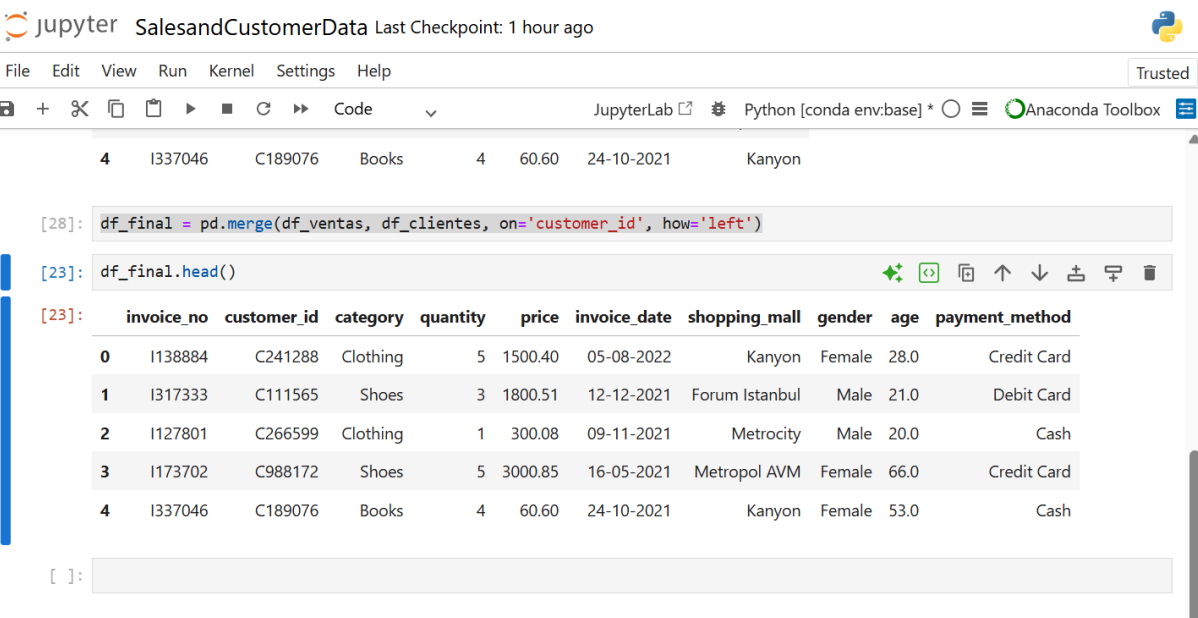
	invoice_no	customer_id	category	quantity	price	invoice_date	shopping_mall
0	I138884	C241288	Clothing	5	1500.40	05-08-2022	Kanyon
1	I317333	C111565	Shoes	3	1800.51	12-12-2021	Forum Istanbul
2	I127801	C266599	Clothing	1	300.08	09-11-2021	Metrocity
3	I173702	C988172	Shoes	5	3000.85	16-05-2021	Metropol AVM
4	I337046	C189076	Books	4	60.60	24-10-2021	Kanyon

2.1.2. Unión de dataframes con información relevante, sin borrar los Dataframes de ventas y clientes: se realizó una unión con *pd.merge()* para combinar las filas en función del campo común *customer_id*, asegurando que cada venta se vincule con el cliente correcto.

Al crear el *df_final* no se borran los dataframes de ventas y clientes.

Para integrar las tablas, se aplicó una unión izquierda (*how='left'*) donde la tabla de Ventas (*df_ventas*) se estableció como la tabla izquierda (principal). Esta elección se justifica en que el objetivo del análisis son las transacciones de venta. Al usar una unión izquierda, aseguramos que ninguna venta sea descartada, incluso si la información detallada del cliente correspondiente estuviera incompleta o faltara en la tabla *df_clientes*. Esto preserva la integridad del conjunto de datos de ventas.

Por último, para validar la unión se utiliza el método *df_final.head()* que nos muestra las primeras 5 filas del nuevo dataframe.



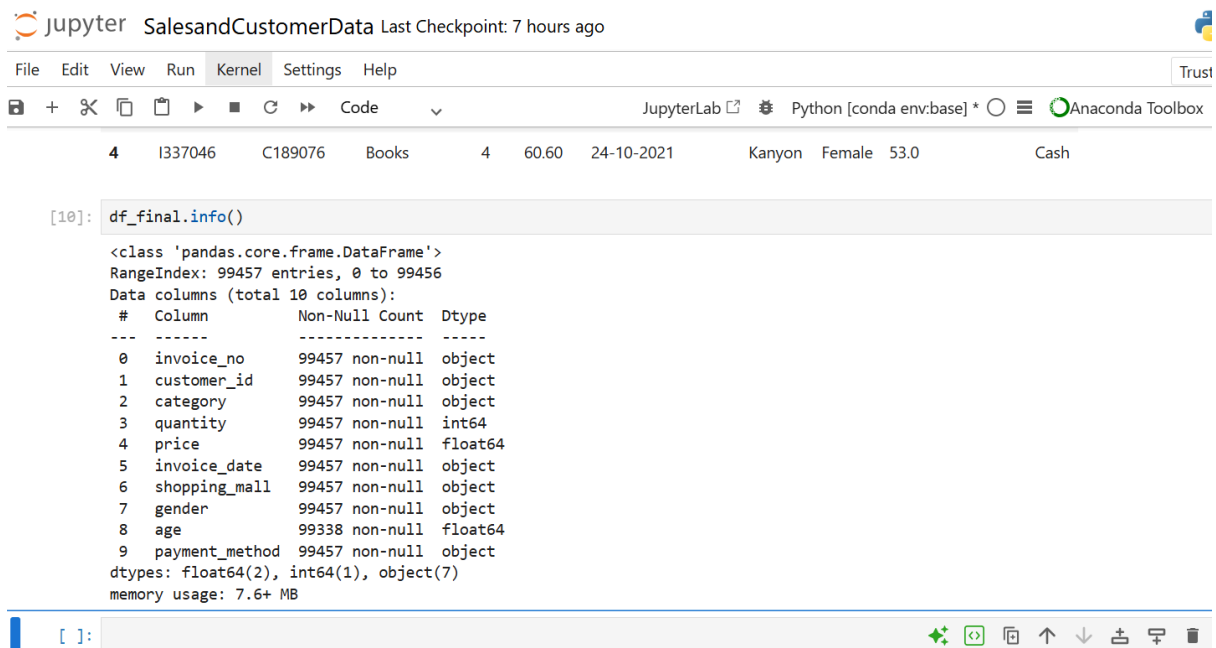
The screenshot shows a JupyterLab window titled "SalesandCustomerData" with a "Trusted" badge. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and code execution. The code editor displays the following Python code:

```
[28]: df_final = pd.merge(df_ventas, df_clientes, on='customer_id', how='left')
[23]: df_final.head()
```

Below the code, the output of `df_final.head()` is shown as a table with 11 columns: *invoice_no*, *customer_id*, *category*, *quantity*, *price*, *invoice_date*, *shopping_mall*, *gender*, *age*, and *payment_method*. The table contains 5 rows of data.

	invoice_no	customer_id	category	quantity	price	invoice_date	shopping_mall	gender	age	payment_method
0	I138884	C241288	Clothing	5	1500.40	05-08-2022	Kanyon	Female	28.0	Credit Card
1	I317333	C111565	Shoes	3	1800.51	12-12-2021	Forum Istanbul	Male	21.0	Debit Card
2	I127801	C266599	Clothing	1	300.08	09-11-2021	Metrocity	Male	20.0	Cash
3	I173702	C988172	Shoes	5	3000.85	16-05-2021	Metropol AVM	Female	66.0	Credit Card
4	I337046	C189076	Books	4	60.60	24-10-2021	Kanyon	Female	53.0	Cash

2.2. Transformación de datos: esta fase se centró en la limpieza, estandarización y preparación del dataframe unido (*df_final*) para el análisis. Las tareas incluyeron la gestión de valores nulos, la estandarización de formatos de fecha y la creación de agrupaciones para responder a las preguntas planteadas en el objetivo. Inmediatamente después de la unión de los dataframes, se realizó un chequeo inicial con el método `.info()` para determinar los tipos de datos iniciales y cuantificar los valores no nulos por columna. Esto fue crucial para planificar la estrategia de limpieza y estandarización, ya que nos alertó sobre la columna *age* con nulos y las columnas de fecha como object, o sea, como cadena de caracteres.



The screenshot shows a JupyterLab window with a file named 'SalesandCustomerData'. The top bar indicates the last checkpoint was 7 hours ago. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and code execution. Below the toolbar, a table displays a single row of data with columns: index, invoice_no, customer_id, category, quantity, price, invoice_date, shopping_mall, gender, age, and payment_method. The values for this row are: 4, I337046, C189076, Books, 4, 60.60, 24-10-2021, Kanyon, Female, 53.0, and Cash. The main area shows a code cell with the command `df_final.info()` executed. The output provides a detailed summary of the DataFrame, including the number of entries (99457), the number of columns (10), and a table of column statistics. The statistics table shows that 7 columns are of type 'object' and 2 are of type 'float64'. The 'age' column is the only one with null values, with 119 missing entries. The total memory usage is reported as 7.6+ MB.

```
[10]: df_final.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99457 entries, 0 to 99456
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype  
---  --
0   invoice_no      99457 non-null  object  
1   customer_id     99457 non-null  object  
2   category        99457 non-null  object  
3   quantity        99457 non-null  int64   
4   price           99457 non-null  float64  
5   invoice_date    99457 non-null  object  
6   shopping_mall   99457 non-null  object  
7   gender          99457 non-null  object  
8   age             99338 non-null  float64  
9   payment_method  99457 non-null  object  
dtypes: float64(2), int64(1), object(7)
memory usage: 7.6+ MB
```

En la captura anterior, podemos observar varias cuestiones importantes:

- El `df_final` tiene 10 columnas en total, 99.457 entradas o registros, la distribución de tipos: object (7), float64 (2) y int64 (1) y **7.6+ MB** de uso de memoria.
- Las columnas numéricas (`quantity`, `price`) son de tipo numérico (int64, float64). Las columnas como `invoice_date` y `payment_method` son de tipo object, lo que implica que la estandarización de la fecha será necesaria para convertirla a `datetime64`. Y por el otro, muestra que la mayoría de las columnas tienen 99457 entradas no nulas, excepto la columna `age`, que tiene 99338 no nulas. Esto indica que quedaron algunos valores nulos en la edad ($99457 - 99338 = 119$ nulos).

El siguiente paso fue gestionar los valores nulos, lo cual es una buena práctica ya que define el tamaño y la composición final del `dataFrame` que se utilizará para todos los análisis subsiguientes. Como los 119 valores nulos en la columna `age` representan el 0.12% del total de las filas, lo cual es un porcentaje ínfimo, lo que hicimos fue utilizar la imputación con la media para preservar la información del 99.88% de las transacciones, logrando rellenar los valores nulos o faltantes por un valor sustituto (la edad promedio de todos los clientes).

En la captura se observa en la línea 12, se calcula la edad promedio y se la muestra en la línea 13. En la línea 15 se usó el método `.fillna()` para rellenar los valores faltantes con la `media_edad`. Y en línea 16 se utilizó el método `.isnull().sum()` para comprobar si la imputación fue

exitosa (el resultado de la ejecución es 0, lo que significa que los 119 valores nulos fueron rellenos con la media_edad)

```
jupyter SalesandCustomerData Last Checkpoint: 41 minutes ago
le Edit View Run Kernel Settings Help Trust
+ 🔍 📄 ▶ ⏏ ↺ ⏩ Code ▾ JupyterLab 🐍 Python [conda env:base] * ⌵ 🟢 Anaconda Toolbox

dtypes: float64(2), int64(1), object(7)
memory usage: 7.6+ MB

[ ]: #Calculo de La Media

[12]: media_edad = df_final['age'].mean()

[13]: media_edad

[13]: np.float64(43.42585918782339)

[ ]: #Relleno de Los valores nulos

[15]: df_final['age'] = df_final['age'].fillna(media_edad)

[16]: df_final['age'].isnull().sum()

[16]: np.int64(0)

[ ]:
```

Luego, se realizó la estandarización de la fecha aplicando la función `pd.to_datetime()` especificando el formato `%d-%m-%Y` para realizar una conversión precisa de la cadena de texto a un formato temporal (`datetime64`). La conversión es fundamental para el análisis exploratorio y garantiza que los datos sean compatibles al ser cargados en la base de datos relacional bajo un tipo de dato adecuado (`DATETIME` o `DATE`).

En la captura se puede ver la conversión en la línea 19 y el resultado en la línea 20 (donde vemos que ahora la columna `invoice_date` es el tipo `datetime64`)

```
[21]: #Convertir La columna de fecha a formato datetime

[19]: df_final['invoice_date'] = pd.to_datetime(df_final['invoice_date'],format='%d-%m-%Y',errors='coerce' #
)

[ ]: # Comprobar La conversion

[20]: df_final.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99457 entries, 0 to 99456
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   invoice_no      99457 non-null  object
1   customer_id     99457 non-null  object
2   category        99457 non-null  object
3   quantity        99457 non-null  int64
4   price           99457 non-null  float64
5   invoice_date    99457 non-null  datetime64[ns]
6   shopping_mall   99457 non-null  object
7   gender          99457 non-null  object
8   age             99457 non-null  float64
9   payment_method  99457 non-null  object
```

A continuación se comienza con las transformaciones analíticas, las cuales podemos agrupar en dos categorías:

- Filtrado: utilizadas básicamente para seleccionar una parte del DataFrame (un subconjunto) para contarlos o analizarlos. Entre estas, se solicita:
 - Métodos de pago realizados por el rango etario de 25 a 35 años.
 - Métodos de pago más utilizados por las mujeres.
- Agrupación: para resumir o calcular algo para cada categoría del DataFrame. Entre las cuales, se pide:
 - Modo de pago más frecuente categorizado por género.
 - Categorización de clientes (por pago, edad y género).
 - Precios por categoría de productos.

Para ir respondiendo a estas cuestiones, se crean los grupos etarios, definiendo los límites de los grupos de edad (bins) y las etiquetas y se crea una columna nueva *age_group*. Se usó el método *.loc* para filtrar las filas y seleccionar la columna de pago para el conteo. Se muestran cuántos clientes hay en cada grupo etario (cuya sumatoria es 99457). Y finalmente, se filtran los métodos de pago dentro del grupo de 25 a 35 años, con lo que se puede responder “Métodos de pagos realizados por el rango etario de 25 a 35 años”

```

7 gender          99457 non-null object
8 age             99457 non-null float64
9 payment_method  99457 non-null object
dtypes: datetime64[ns](1), float64(2), int64(1), object(6)
memory usage: 7.6+ MB

```

```
[ ]: # Creación de grupos etarios, definiendo límites y etiquetas de cada uno
```

```
[22]: bins = [18, 25, 36, 50, 65, 100]
labels = ['18-24', '25-35', '36-49', '50-64', '65+']
```

```
[23]: df_final['age_group'] = pd.cut(
    df_final['age'],
    bins=bins,
    labels=labels,
    right=False,
    include_lowest=True
)
```

```
[26]: df_final[['age', 'age_group']].head(10)
```

```
[26]:
```

	age	age_group
0	28.0	25-35
1	21.0	18-24
2	20.0	18-24
3	66.0	65+
4	53.0	50-64
5	28.0	25-35
6	49.0	36-49
7	32.0	25-35
8	69.0	65+
9	60.0	50-64

```
[27]: metodos_pago_25_35 = (
    df_final.loc[df_final['age_group'] == '25-35', 'payment_method']
    .value_counts()
)
```

```
[28]: df_final['age_group'].value_counts()
```

```
[28]:
```

age_group	count
50-64	28515
36-49	27155
25-35	20893
18-24	13484
65+	9410

Name: count, dtype: int64

```
[ ]: #Métodos de pago grupo etario 25-35
```

```
[27]: metodos_pago_25_35 = (
    df_final.loc[df_final['age_group'] == '25-35', 'payment_method']
    .value_counts()
)
```

```
[25]: #Métodos de pago grupo etario 25-35
```

```
[26]: metodos_pago_25_35 = (
    df_final.loc[df_final['age_group'] == '25-35', 'payment_method']
    .value_counts()
)
```

```
[27]: print(metodos_pago_25_35)
```

```

payment_method
Cash          9356
Credit Card  7274
Debit Card   4263
Name: count, dtype: int64

```

Para conocer los métodos de pago más utilizados por las mujeres, se recurre al método `.loc` para filtrar las filas donde el género es 'Female'. Luego, se aplica `.value_counts()` a la columna `payment_method` para obtener la frecuencia de uso.


```
[36]: #Métodos de pago grupo etario Mujeres

[37]: metodos_pago_mujeres = (
      df_final.loc[df_final['gender'] == 'Female', 'payment_method']
      .value_counts()
    )

[38]: print(metodos_pago_mujeres)

payment_method
Cash          26509
Credit Card  21011
Debit Card    11962
Name: count, dtype: int64
```

Se determinó el modo de pago más frecuente de todos los clientes (cuyo resultado es cash) con el método `.mode()` que devuelve el valor más repetido.

```
[39]: #Métodos de pago mas frecuente de todos los clientes

[40]: moda_general = df_final['payment_method'].mode()[0]

[41]: print(moda_general)

Cash
```

Para calcular el método de pago más frecuente de todos los clientes categorizados por género, se utilizó la función `.groupby('gender')` para segmentar el *DataFrame* en dos subconjuntos (Male y Female). Luego, se aplicó la función de agregación `.agg(pd.Series.mode)` sobre la columna `payment_method`. Este método calcula la *moda* independientemente para cada grupo de género, proporcionando la respuesta directa a la consigna.

```
*[42]: # Métodos de pago más frecuente de todos los clientes, categorizados por género

[43]: moda_por_genero = df_final.groupby('gender')['payment_method'].agg(pd.Series.mode)

[44]: print(moda_por_genero)

gender
Female    Cash
Male      Cash
Name: payment_method, dtype: object
```

También se solicita una categorización de clientes de acuerdo a su forma de pago (por edad y género). Para esto se empleó la técnica de agrupamiento multinivel utilizando `.groupby()` con una lista de las tres variables clave. La función `.size()` calcula el conteo de elementos dentro de cada combinación de grupos. Finalmente, se utiliza `.reset_index()` para convertir el resultado en un *DataFrame* legible, con una columna explícita (*Total_Clientes*) para el conteo de cada segmento.

```
[45]: # Categorización de clientes de acuerdo a su forma de pago (por edad y por genero)

[49]: categorizacion = (
    df_final.groupby(['age_group', 'gender', 'payment_method'], observed=True)
    .size()
    .reset_index(name='Total_Clientes')
    .sort_values(by='Total_Clientes', ascending=False)
)

[50]: print(categorizacion.head(10))
```

	age_group	gender	payment_method	Total_Clientes
18	50-64	Female	Cash	7639
12	36-49	Female	Cash	7262
19	50-64	Female	Credit Card	5995
13	36-49	Female	Credit Card	5803
6	25-35	Female	Cash	5521
21	50-64	Male	Cash	5136
15	36-49	Male	Cash	4832
7	25-35	Female	Credit Card	4401
22	50-64	Male	Credit Card	3979
9	25-35	Male	Cash	3835

```
[ ]:
```

Por último, para visualizar los precios por categoría de productos, se aplicó `.groupby('category')` para segmentar los datos por tipo de producto. La función `.describe()` es una herramienta de Análisis Exploratorio de Datos (EDA) que proporciona un resumen estadístico completo (incluyendo el conteo, media, desviación estándar, valores mínimo, máximo y los cuartiles). Esto permite comparar rápidamente la distribución de precios entre las diferentes categorías de productos.

```
[51]: # Precios por categoría de productos

[52]: analisis_precios = df_final.groupby('category')['price'].describe()

[53]: print(analisis_precios)
```

	count	mean	std	min	25%	50% \
category						
Books	4981.0	45.568621	21.492662	15.15	30.30	45.45
Clothing	34487.0	901.084021	424.225594	300.00	600.16	900.24
Cosmetics	15097.0	122.448626	57.458097	40.66	81.32	121.98
Food & Beverage	14776.0	15.671948	7.422176	5.23	10.46	15.69
Shoes	10034.0	1807.388568	845.116269	600.17	1200.34	1800.51
Souvenir	4999.0	34.894345	16.564498	11.73	23.46	35.19
Technology	4996.0	3156.935548	1481.559805	1050.00	2100.00	3150.00
Toys	10087.0	107.733185	50.364844	35.84	71.68	107.52

	75%	max
category		
Books	60.60	75.75
Clothing	1200.32	1500.40
Cosmetics	162.64	203.30
Food & Beverage	20.92	26.15
Shoes	2400.68	3000.85
Souvenir	46.92	58.65
Technology	4200.00	5250.00
Toys	143.36	179.20

```
[ ]:
```

2.3. Limpieza de datos: El nuevo DataFrame, denominado `df_limpio`, se creó como una copia explícita de `df_final` al finalizar todas las etapas de la Transformación. Este DataFrame está listo para la carga (Load), ya que se ha garantizado: la imputación de nulos en la columna `age`, la

creación de variables categóricas (age_group, month_name, etc.) y la conversión de invoice_date al tipo de dato datetime64.

```
df_final.head()

[101]: invoice_no customer_id category quantity price invoice_date shopping_mall gender age payment_method age_group month_name day_name
0 I138884 C241288 Clothing 5 1500.40 2022-08-05 Kanyon Female 28.0 Credit Card 25-35 August Friday
1 I317333 C111565 Shoes 3 1800.51 2021-12-12 Forum Istanbul Male 21.0 Debit Card 18-24 December Sunday
2 I127801 C266599 Clothing 1 300.08 2021-11-09 Metrocity Male 20.0 Cash 18-24 November Tuesday
3 I173702 C988172 Shoes 5 3000.85 2021-05-16 Metropol AvM Female 66.0 Credit Card 65+ May Sunday
4 I337046 C189076 Books 4 60.60 2021-10-24 Kanyon Female 53.0 Cash 50-64 October Sunday

[102]: df_limpio = df_final.copy()

[103]: df_limpio.head()

[103]: invoice_no customer_id category quantity price invoice_date shopping_mall gender age payment_method age_group month_name day_name
0 I138884 C241288 Clothing 5 1500.40 2022-08-05 Kanyon Female 28.0 Credit Card 25-35 August Friday
1 I317333 C111565 Shoes 3 1800.51 2021-12-12 Forum Istanbul Male 21.0 Debit Card 18-24 December Sunday
2 I127801 C266599 Clothing 1 300.08 2021-11-09 Metrocity Male 20.0 Cash 18-24 November Tuesday
3 I173702 C988172 Shoes 5 3000.85 2021-05-16 Metropol AvM Female 66.0 Credit Card 65+ May Sunday
4 I337046 C189076 Books 4 60.60 2021-10-24 Kanyon Female 53.0 Cash 50-64 October Sunday

[*]: df_final.to_excel('informe_final_limpio.xlsx', index=False)

[ ]:
```

Como se puede ver en la imagen anterior y a modo de finalización del ETL, se lo exporta a formato excel para tener una backup legible.

2.4. Carga de datos y consultas SQL:

El primer paso fue instalar el conector de Python que permite a Pandas comunicarse con MySQL:

- *sqlalchemy*: es la librería que permite a Pandas interactuar con diferentes motores de bases de datos.
- *mysql-connector-python* o *pymysql*: Son los *drivers* que MySQL necesita para la conexión.

```
# Carga final a MySQL

!pip install sqlalchemy mysql-connector-python pymysql

Requirement already satisfied: sqlalchemy in c:\users\todop\anaconda3\lib\site-packages (2.0.39)
Collecting mysql-connector-python
  Downloading mysql_connector_python-9.4.0-cp313-cp313-win_amd64.whl.metadata (7.7 kB)
Collecting pymysql
  Downloading pymysql-1.1.2-py3-none-any.whl.metadata (4.3 kB)
Requirement already satisfied: greenlet!=0.4.17 in c:\users\todop\anaconda3\lib\site-packages (from sqlalchemy) (3.1.1)
Requirement already satisfied: typing-extensions>=4.6.0 in c:\users\todop\anaconda3\lib\site-packages (from sqlalchemy) (4.12.2)
Downloading mysql_connector_python-9.4.0-cp313-cp313-win_amd64.whl (16.4 MB)
----- 0.0/16.4 MB ? eta -:--:--
----- 0.3/16.4 MB ? eta -:--:--
----- 1.0/16.4 MB 2.8 MB/s eta 0:00:06
----- 1.6/16.4 MB 2.8 MB/s eta 0:00:06
----- 2.1/16.4 MB 2.8 MB/s eta 0:00:06
----- 2.6/16.4 MB 2.7 MB/s eta 0:00:06
----- 3.1/16.4 MB 2.7 MB/s eta 0:00:05
----- 3.9/16.4 MB 2.7 MB/s eta 0:00:05
----- 4.5/16.4 MB 2.7 MB/s eta 0:00:05
----- 5.0/16.4 MB 2.7 MB/s eta 0:00:05
```

Luego configuramos la conexión en Jupyter creando una "cadena de conexión" con las credenciales de MySQL.

```
jupyter ABP_SalesandCustomerData Last Checkpoint: 18 minutes ago

Edit View Run Kernel Settings Help

+ ✂ 📄 📄 ▶ ⏮ ⏭ Code ⌵ JupyterLab 🌐 Python [conda env:base] * ○ ≡ 🐍 Anaconda Toolbox

[68]: import pandas as pd
      from sqlalchemy import create_engine
      import getpass # Para ingresar la contraseña de forma segura

      # --- Variables de Conexión ---
      USUARIO = 'root'
      PASSWORD = getpass.getpass('root: ') # Ingresa tu contraseña de forma segura
      HOST = 'localhost' # O La IP de tu servidor MySQL
      PUERTO = 3306 # Puerto estándar de MySQL
      BASE_DE_DATOS = 'ABP_datos_limpios' # Debe existir previamente en MySQL

      # Crear la cadena de conexión (URL de SQLAlchemy)
      # EL formato es: 'dialect+driver://user:password@host:port/database'
      CONN_STRING = f"mysql+pymysql://{USUARIO}:{PASSWORD}@{HOST}:{PUERTO}/{BASE_DE_DATOS}"

      # Crear el motor de conexión
      engine = create_engine(CONN_STRING)

      print("Conexión con el motor SQLAlchemy creada.")

root: .....
Conexión con el motor SQLAlchemy creada.
```

Una vez creada la conexión, cargamos el DataFrame a MySQL. Finalmente, usamos el método `.to_sql()` de Pandas, apuntando al motor de MySQL que creaste.

```
[90]: # Carga del DataFrame a MySQL

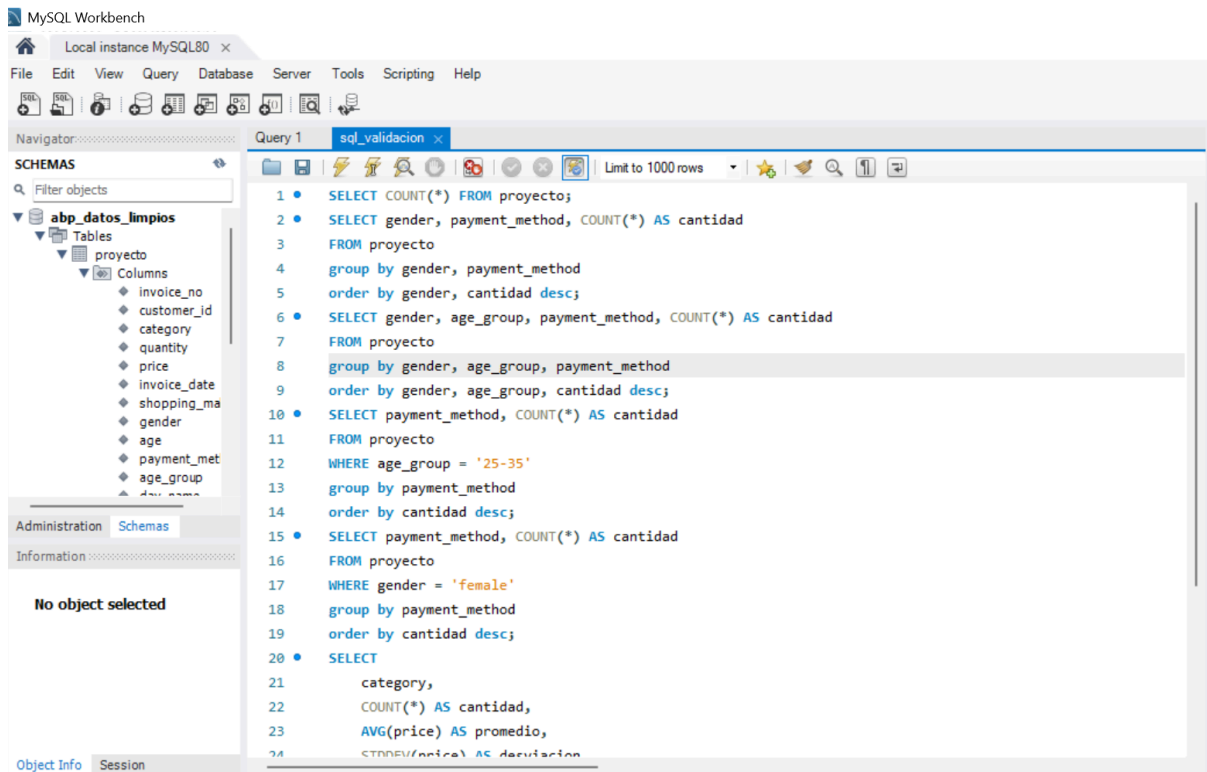
[91]: NOMBRE_TABLA = 'proyecto' # Nombre que tendrá la tabla en MySQL

      try:
          df_final.to_sql(
              name=NOMBRE_TABLA,
              con=engine,
              if_exists='replace', # 'replace' borra y recrea la tabla si ya existe
              index=False,        # No carga el índice de Pandas como una columna
              )
          print(f"✅ ¡Carga exitosa! Datos cargados a la tabla '{NOMBRE_TABLA}' en MySQL.")

      except Exception as e:
          print(f"❌ Error al cargar los datos: {e}")

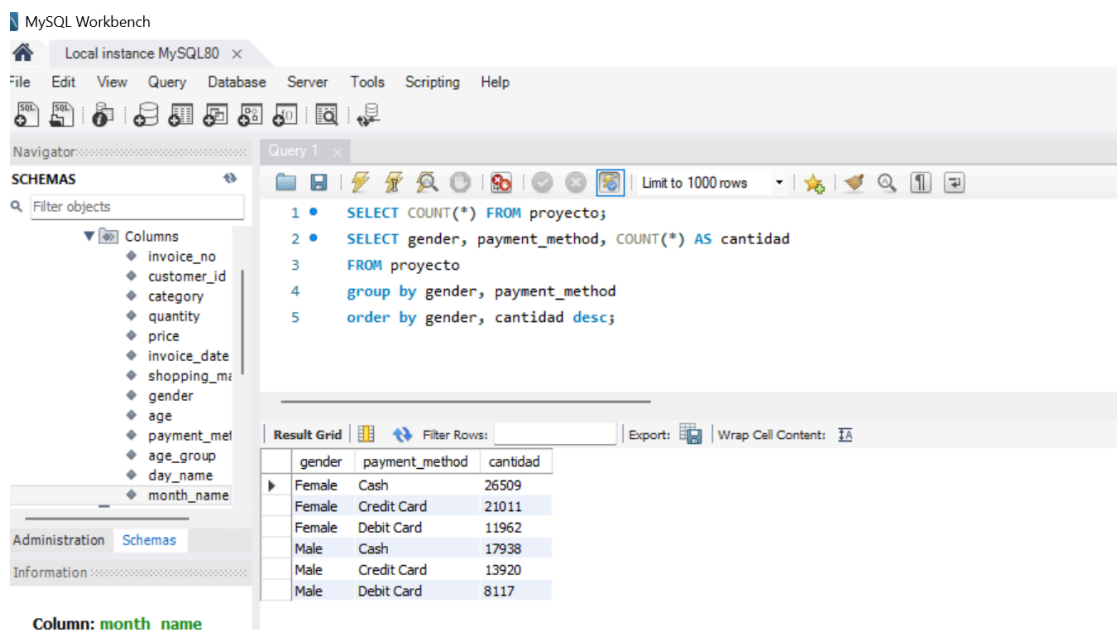
✅ ¡Carga exitosa! Datos cargados a la tabla 'proyecto' en MySQL.
```

Al abrir MySQL Workbench, se puede ver la base de datos abp_datos_limpios con la tabla proyecto que contiene toda la información del dataframe final y limpio.



A continuación se pueden ir viendo las **consultas SQL de validación**:

Modo de pago más frecuente de todos los clientes y a su vez categorizados por género: Se agruparon los datos por gender y payment_method, y se contó la cantidad de operaciones con COUNT (*). Luego se ordenaron los resultados para identificar el método más utilizado por cada grupo.



Categorización de clientes de acuerdo a su forma de pago (por edad y género): Se realizó una categorización de clientes según su método de pago, agrupando por género y grupo de edad (age_group). Se utilizó COUNT(*) para calcular la cantidad de operaciones en cada combinación. La consulta se ordenó por género y grupo de edad para facilitar la lectura de los resultados.

Query 1 x

```

2 • SELECT gender, payment_method, COUNT(*) AS cantidad
3 FROM proyecto
4 group by gender, payment_method
5 order by gender, cantidad desc;
6 • SELECT gender, age_group, payment_method, COUNT(*) AS cantidad
7 FROM proyecto
8 group by gender, age_group, payment_method
9 order by gender, age_group, cantidad desc;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

gender	age_group	payment_method	cantidad
Female	18-24	Cash	3524
Female	18-24	Credit Card	2843
Female	18-24	Debit Card	1564
Female	25-35	Cash	5521
Female	25-35	Credit Card	4401
Female	25-35	Debit Card	2613
Female	36-49	Cash	7262
Female	36-49	Credit Card	5803
Female	36-49	Debit Card	3261
Female	50-64	Cash	7639

Result 16 x

Métodos de pagos realizados por el rango etario de 25 a 35 años: Se realizó una consulta para identificar los métodos de pago más utilizados por clientes del rango etario de **25 a 35 años**. Se filtraron los datos con age_group = '25-35', agrupando por payment_method y contando la cantidad de operaciones con COUNT(*).

Query 1 x

```

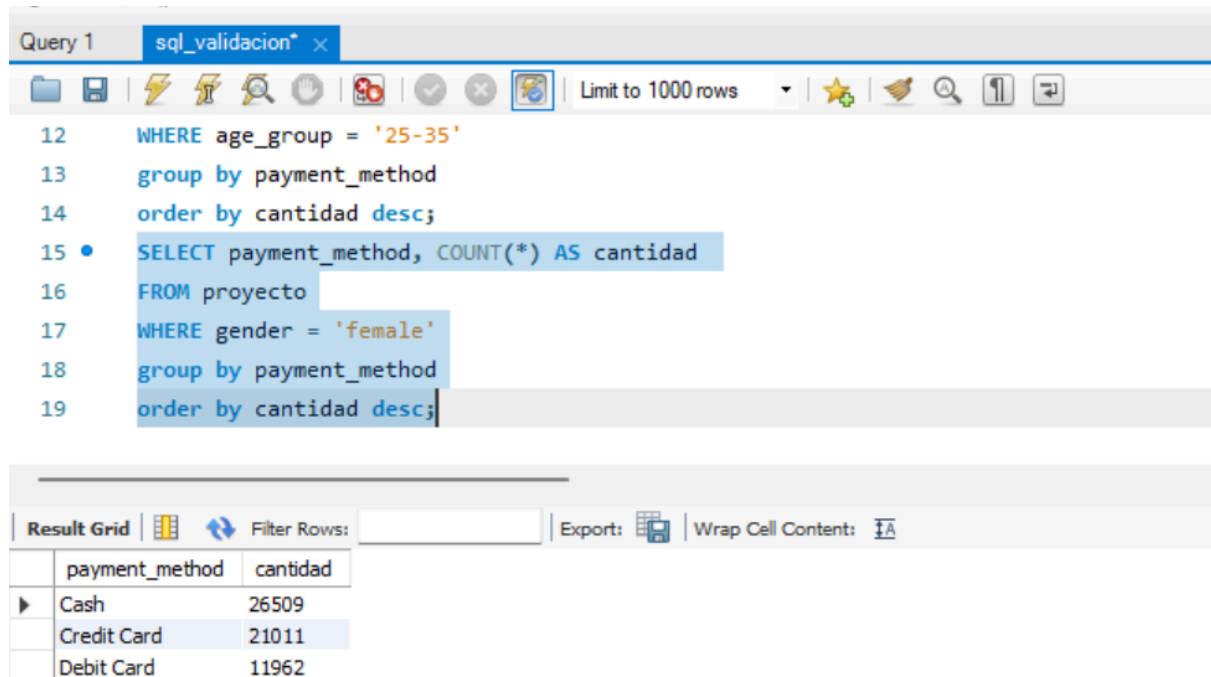
7 FROM proyecto
8 group by gender, age_group, payment_method
9 order by gender, age_group, cantidad desc;
10 • SELECT payment_method, COUNT(*) AS cantidad
11 FROM proyecto
12 WHERE age_group = '25-35'
13 group by payment_method
14 order by cantidad desc;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

payment_method	cantidad
Cash	9356
Credit Card	7274
Debit Card	4263

Métodos de pago más utilizados por las mujeres: Se realizó una consulta para identificar los métodos de pago más utilizados por clientes de género femenino. Se filtraron los datos con `gender = 'Female'`, agrupando por `payment_method` y contando la cantidad de operaciones con `COUNT(*)`.



The screenshot displays a SQL query in a tool window titled 'Query 1' with the file name 'sql_validacion*.x'. The query is as follows:

```
12 WHERE age_group = '25-35'
13 group by payment_method
14 order by cantidad desc;
15 • SELECT payment_method, COUNT(*) AS cantidad
16 FROM proyecto
17 WHERE gender = 'female'
18 group by payment_method
19 order by cantidad desc;
```

Below the query editor, the 'Result Grid' shows the results of the query. It includes a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox. The data is presented in a table with two columns: 'payment_method' and 'cantidad'.

payment_method	cantidad
Cash	26509
Credit Card	21011
Debit Card	11962

Precios por categoría de producto: Para cada categoría, se calcularon métricas descriptivas como la cantidad de registros (`COUNT`), el precio promedio (`AVG`), la desviación estándar (`STDDEV`), el precio mínimo (`MIN`) y el precio máximo (`MAX`). Los resultados se ordenaron por cantidad de registros en orden descendente (`ORDER BY cantidad DESC`) para destacar las categorías con mayor volumen de operaciones. Este enfoque facilita la comparación entre categorías y aporta información útil para decisiones comerciales, segmentación de clientes o análisis de rentabilidad.

Query 1 sql_validacion* x

Limit to 1000 rows

```

20 • SELECT
21     category,
22     COUNT(*) AS cantidad,
23     AVG(price) AS promedio,
24     STDDEV(price) AS desviacion,
25     MIN(price) AS minimo,
26     MAX(price) AS maximo
27 FROM
28     proyecto
29 GROUP BY
30     category
31 ORDER BY
32     cantidad DESC;

```

Result Grid Filter Rows: Export: Wrap Cell Content:

	category	cantidad	promedio	desviacion	minimo	maximo
▶	Clothing	34487	901.0840212250755	424.21944298435363	300.08	1500.4
	Cosmetics	15097	122.4486255547387	57.45619411717906	40.66	203.3
	Food & Beverage	14776	15.671948429887172	7.42192455892758	5.23	26.15
	Toys	10087	107.73318528799395	50.36234730715971	35.84	179.2
	Shoes	10034	1807.3885678691547	845.0741558068648	600.17	3000.85
	Souvenir	4999	34.89434486897436	16.562841352516255	11.73	58.65
	Technology	4996	3156.935548438751	1481.4115230852976	1050	5250
	Books	4981	45.56862075888369	21.49050452798674	15.15	75.75

Result 2 x

CONCLUSIÓN FINAL DEL ETL: Todas las consultas SQL dan los mismos resultados que los cálculos en Jupyter, por lo que es posible concluir que la fase de Carga (L de ETL) y la Validación de Datos son perfectas. Hemos logrado dos cosas: la consistencia de datos (los datos limpios y transformados en Pandas fueron transferidos sin errores al entorno MySQL) y una validación robusta (la información obtenida en el entorno de análisis Jupyter es idéntica a la del entorno de persistencia MySQL Workbench).